

- Description

This module manages an online exam grading and ranking system, ensuring that:

1. **Students' scores are recorded and must be within the range of 0 to 100.**
2. **Students are ranked** based on their scores in descending order.
3. **The top scorer(s) receive a distinction** if their score exceeds a threshold (e.g., 90).
4. **The system maintains a history of scores and ranks** for future analysis.

This system ensures fair grading, automated ranking, and recognition of top performers.

- Specification

module OnlineExamGrading

var scores: sequence of (string, real)

var rankingHistory: sequence of string

process recordScore(name: string, score: real) flag: boolean

functional scenario form:

score ≥ 0 and score ≤ 90 $\&\&$ flag = true and scores = scores $\oplus$ (name, score)

and rankingHistory = rankingHistory $\oplus$ ("Recorded: " + name + " - " + score), Valid score, recorded successfully

||

score > 90 and score ≤ 100 $\&\&$ flag = true and scores = scores $\oplus$ (name, score) and

rankingHistory = rankingHistory $\oplus$ ("Recorded: " + name + " - " + score), Valid score, recorded successfully

||

score < 0 || score > 100 $\&\&$ flag = false, Invalid score, rejected

end process

process rankStudents()

functional scenario form:

(|scores| > 0) $\&\&$ scores = sort(scores, descending) $\wedge$ rankingHistory =

rankingHistory $\oplus$ "Ranking updated", Students ranked successfully

end process

end module

- Java Code

```
package code;
import java.util.*;
class Student {
    private String name;
    private double score;
    public Student(String name, double score) {
        this.name = name;
        this.score = score;
    }
}
```

```

    public String getName() {
        return name;
    }
    public double getScore() {
        return score;
    }
}

class ExamSystem {
    private List<Student> students;
    private List<String> rankingHistory;
    public ExamSystem() {
        this.students = new ArrayList<>();
        this.rankingHistory = new ArrayList<>();
    }
    public boolean recordScore(String name, double score) {
        if (score >= 0 && score <= 100) {
            students.add(new Student(name, score));
            rankingHistory.add(String.format("Recorded: %s - %.1f", name, score));
            System.out.printf("Score recorded: %s - %.1f\n", name, score);
            return true;
        } else {
            System.out.println("Invalid score! Must be between 0 and 100.");
            return false;
        }
    }
    public void rankStudents() {
        students.sort((s1, s2) -> Double.compare(s2.getScore(), s1.getScore()));
        System.out.println("\nExam Rankings:");
        for (int i = 0; i < students.size(); i++) {
            String distinction = (students.get(i).getScore() > 90) ? "(Distinction)" : "";
            System.out.printf("%d. %s - %.1f%s\n", i + 1, students.get(i).getName(), students.get(i).getScore(), distinction);
            rankingHistory.add(String.format("Ranked: %d. %s - %.1f%s", i + 1, students.get(i).getName(), students.get(i).getScore(), distinction));
        }
    }
    public void printRankingHistory() {
        System.out.println("\nRanking History:");
    }
}

```

```

        for (String log : rankingHistory) {
            System.out.println(log);
        }
    }
}

public class c10_OnlineExamSystem {
    public static void main(String[] args) {
        ExamSystem exam = new ExamSystem();
        // testcase-VT:
        //     exam.recordScore("Alice", -1.53);    // score < 0
        //     exam.recordScore("Bob", 2.78);        // score >= 0 && score <= 90
        //     exam.recordScore("Charlie", 92.64);    // score > 90 && score <=
100
        //     exam.recordScore("David", 104.22);    // score > 100
        // testcase-FT:
        //     exam.recordScore("Alice", 7.022238808055921E305);
        //     exam.recordScore("Bob", 1.7976931348623157E308);
        //     exam.recordScore("Charlie", 0.0);
        //     exam.recordScore("David", -2.2250738585072014E-308);
        // testcase-Z3:
        //     exam.recordScore("Alice", 90);    // score < 0
        //     exam.recordScore("Bob", 79);        // score >= 0 && score <= 90
        //     exam.recordScore("Charlie", 101);    // score > 90 && score <= 10
0
        //     exam.recordScore("David", 79);        // score > 100

        // testcase-UVT:
        exam.recordScore("Alice", 90);    // score < 0
        exam.recordScore("Bob", 79);        // score >= 0 && score <= 90
        exam.recordScore("Charlie", 101);    // score > 90 && score <= 100
        exam.recordScore("David", -7);        // score > 100

        exam.rankStudents();
        exam.printRankingHistory();
    }
}

```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	818	828
✓ src	1.2 %	10	818	828
✓ code	1.2 %	10	818	828
✓ c10_OnlineExamSystem.java	100.0 %	10	0	10
> c10_OnlineExamSystem		0	0	0
> ExamSystem	100.0 %	10	0	10
> Student		0	0	0

3. Vibration testing instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.5 %	242	15,375	15,617
✓ src	1.5 %	242	15,375	15,617
✓ code	1.5 %	242	15,375	15,617
✓ c10_OnlineExamSystem.java	98.8 %	242	3	245
> c10_OnlineExamSystem	90.6 %	29	3	32
> ExamSystem	100.0 %	198	0	198
> Student	100.0 %	15	0	15

4. Fuzz testing branch coverage: 90%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.1 %	9	819	828
✓ src	1.1 %	9	819	828
✓ code	1.1 %	9	819	828
✓ c10_OnlineExamSystem.java	90.0 %	9	1	10
> ExamSystem	90.0 %	9	1	10
> c10_OnlineExamSystem		0	0	0
> Student		0	0	0

5. Fuzz testing instruction coverage: 95.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.5 %	234	15,383	15,617
✓ src	1.5 %	234	15,383	15,617
✓ code	1.5 %	234	15,383	15,617
✓ c10_OnlineExamSystem.java	95.5 %	234	11	245
> ExamSystem	96.0 %	190	8	198
> c10_OnlineExamSystem	90.6 %	29	3	32
> Student	100.0 %	15	0	15

6. Z3 branch coverage: 80%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	832	840
✓ src	1.0 %	8	832	840
✓ code	1.0 %	8	832	840
✓ c10_OnlineExamSystem.java	80.0 %	8	2	10
> ExamSystem	80.0 %	8	2	10
> c10_OnlineExamSystem		0	0	0
> Student		0	0	0

7. Z3 instruction coverage: 98%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.1 %	240	21,853	22,093
✓ src	1.1 %	240	21,853	22,093
✓ code	1.1 %	240	21,853	22,093
✓ c10_OnlineExamSystem.java	98.0 %	240	5	245
> c10_OnlineExamSystem	90.6 %	29	3	32
> ExamSystem	99.0 %	196	2	198
> Student	100.0 %	15	0	15

8. Unimproved V-method branch coverage: 90%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.1 %	9	831	840
✓ src	1.1 %	9	831	840
✓ code	1.1 %	9	831	840
✓ c10_OnlineExamSystem.java	90.0 %	9	1	10
> ExamSystem	90.0 %	9	1	10
> c10_OnlineExamSystem		0	0	0
> Student		0	0	0

9. Unimproved V-method instruction coverage: 98%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼  VibrationTestingTool	 1.1 %	9	831	840
▼  src	 1.1 %	9	831	840
▼  code	 1.1 %	9	831	840
▼  c10_OnlineExamSystem.java	 90.0 %	9	1	10
>  ExamSystem	 90.0 %	9	1	10
>  c10_OnlineExamSystem		0	0	0
>  Student		0	0	0